

Modern C++ Topics: Static Reflection

Alex Cui
alexcui03@sjtu.edu.cn

2026-04-25

SJTU *NIX User Group

Outline

Introduction	1	Limited Code Generation	28
Language and Library Features	3	Compile-Time Variables	30
Expansion Statement (Template-For)	4	Applications	31
“Cat-Ears” Operator (^)	6	Parsing Command Lines	32
Splicers ([: :])	10	Serialization and Deserialization	33
std::meta::info	12	MVVM in C++	34
Meta Library	13		
define_aggregate	15		
Reflection with Types and Templates . .	17		
Compile-Time Strings	21		
Annotations	24		
Compile-Time Programming	26		
constexpr and constexpr	27		

Reflection

The ability to **examine**, **introspect**, and **modify** its own structure and behavior.

Reflection

The ability to **examine**, **introspect**, and **modify** its own structure and behavior.

Static Reflection: Inspect types at compile time with templates and type traits.

The following example is taken from reflect-cpp:

```
struct Person {
    std::string first_name;
    std::string last_name;
    int age;
};

const auto homer = Person{ .first_name = "Homer", .last_name = "Simpson",
    .age = 45 };

// We can now write into and read from a JSON string.
const std::string json_string = rfl::json::write(homer);
auto homer2 = rfl::json::read<Person>(json_string).value();
```

Reflection

The ability to **examine**, **introspect**, and **modify** its own structure and behavior.

Dynamic Reflection: Introspect and modify an object at runtime.

```
struct MyStruct { MyStruct() {}; void func(double) {}; int data; };
```

```
RTTR_REGISTRATION {  
    registration::class_<MyStruct>("MyStruct")  
        .constructor<>()  
        .property("data", &MyStruct::data)  
        .method("func", &MyStruct::func);  
}
```

```
type t = type::get<MyStruct>();  
for (auto& prop : t.get_properties())  
    std::cout << "name: " << prop.get_name();
```

Outline

Introduction	1	Limited Code Generation	28
Language and Library Features	3	Compile-Time Variables	30
Expansion Statement (Template-For)	4	Applications	31
“Cat-Ears” Operator (^)	6	Parsing Command Lines	32
Splicers ([:])	10	Serialization and Deserialization	33
std::meta::info	12	MVVM in C++	34
Meta Library	13		
define_aggregate	15		
Reflection with Types and Templates . .	17		
Compile-Time Strings	21		
Annotations	24		
Compile-Time Programming	26		
constexpr and constexpr	27		

Expansion Statement (Template-For)

Expansion statement enables the **compile-time repetition** of a statement for each element of iterable objects.

```
template for (auto x : {1, 'a', 1.0}) {  
    std::cout << x << std::endl;  
}
```

The above code will be compiled into:

```
{ auto x = 1; std::cout << x << std::endl; }  
{ auto x = 'a'; std::cout << x << std::endl; }  
{ auto x = 1.0; std::cout << x << std::endl; }
```

continue and break are **not allowed** in expansion statements.

Expansion Statement (Template-For)

Example: Traversing a tuple.

```
print_tuple(std::make_tuple(1, 'a', 1.5));
```

Expansion Statement (Template-For)

Example: Traversing a tuple.

```
print_tuple(std::make_tuple(1, 'a', 1.5));
```

Before C++ 17:

```
template <std::size_t I = 0, typename... Tp>  
typename std::enable_if<I == sizeof...(Tp), void>::type  
print_tuple(const std::tuple<Tp...> &) {}
```

```
template <std::size_t I = 0, typename... Tp>  
typename std::enable_if<I < sizeof...(Tp), void>::type  
print_tuple(const std::tuple<Tp...> &t) {  
    std::cout << std::get<I>(t) << std::endl;  
    print_tuple<I + 1>(t);  
}
```

Expansion Statement (Template-For)

Example: Traversing a tuple.

```
print_tuple(std::make_tuple(1, 'a', 1.5));
```

After C++ 17:

```
template <typename Tp>
void print_tuple(const Tp &t) {
    std::apply([](auto &&...args) {
        ((std::cout << args
            << std::endl), ...);
    }, t);
}
```

After C++ 26:

```
template <typename Tp>
void print_tuple(const Tp &t) {
    template for (auto x : t) {
        std::cout << x << std::endl;
    }
}
```

“Cat-Ears” Operator (^)

“^” operator produces a **reflection value** from its operand.

- takes a *namespace-name*, a *type-id*, or an *id-expression*
- returns a `std::meta::info` of the reflection
- return value must be held by a **constexpr** variable

```
constexpr auto refl_global = ^::;           // reflect the global namespace
constexpr auto refl_type = ^int;           // reflection of int type
```

```
int x;
constexpr auto refl_var = ^x;              // reflection of variable x
constexpr auto refl_type_x = ^decltype(x);
```

```
// auto not_constexpr = ^::;              // Syntax Error!
```

“Cat-Ears” Operator (^)

An *id-expression* can be one of:

- variable, static data member, or structured binding
- function (including member functions)
- non-static data member
- template or member template
- enumerator

```
struct A { int x; static int y; };
```

```
template <class T>  
struct B { using type = T; };
```

```
enum E { EA, EB };  
void foo();  
A obj;
```

```
constexpr auto r1 = ^^EA;  
constexpr auto r2 = ^^foo;  
constexpr auto r3 = ^^A;  
constexpr auto r4 = ^^A::x;  
constexpr auto r5 = ^^A::y;  
constexpr auto r6 = ^^obj;  
constexpr auto r7 = ^^B;  
constexpr auto r8 = ^^B<int>;  
constexpr auto r9 = ^^B<int>::type;  
// constexpr auto r = ^^0;    // Error
```

“Cat-Ears” Operator (^)

We cannot reflect a value or an object by ^ operator.

<meta> provides several utility functions to reflect particular values or objects.

```
#include <meta>
```

```
int x, arr[10];    // global variables
```

```
constexpr auto r1 = std::meta::reflect_object(x);
```

```
// Cannot use ^^arr[1]
```

```
constexpr auto r2 = std::meta::reflect_object(arr[1]);
```

```
constexpr auto r3 = std::meta::reflect_object(arr);
```

```
// Cannot use ^^arr[1];
```

```
constexpr auto r4 = std::meta::reflect_constant(42);
```

“Cat-Ears” Operator (^)

Why not `reflexpr`?

Introduced initially, but it is too “heavy” and insufficiently distinct from a function call.

Why not `^`?

`^` is already used as a binary operator of XOR.

For compatibility with Microsoft C++/CLI and Apple Objective-C.

What’s next?

Introduce `^(expression)` in the future.

Splicers ([: :])

A reflection can be “spliced” into source code.

- [: r :] produces an **expression** evaluating to the entity represented by *r*, if permitted.

```
int x;  
[ : ^x : ] = 10;
```

- typename[: r :] and template[: r :] to produces a **type** or **template** name.

```
constexpr auto r = ^int;  
typename[ : r : ] x = 10;    // int x = 10;  
typename[ : ^double : ] y;  // double y;
```

- [: r :]:: produces a **nested name**.

```
struct A { static int x; };  
constexpr auto r = ^A;  
[ : r : ]::x = 10;
```

Splicers ([: :])

The operand of splicers will be implicitly converted to a **std::meta::info prvalue**, and must be an **constexpr** value.

If the reflection does not meet the requirement, it is **ill-formed**.

```
// typename[: ^^:: :] x = 0;    // Error!
```

Splicing namespaces in namespace definitions is not allowed.

```
namespace A {}
constexpr std::meta::info NS_A = ^^A;

namespace B {
    namespace [:NS_A:] {
        void fn(); // Is this '::A::fn' or '::B::A::fn' ?
    }
}
```

std::meta::info

std::meta::info is the type representing the reflection.

```
namespace std {  
    namespace meta {  
        using info = decltype(^^::);  
    }  
}
```

- std::meta::info **not** be allowed to propagate to **runtime**.
- all types compounded from it should also be consteval-only. (e.g. vector<info>, info *)
- default constructor produces a **null reflection value**.

```
struct S {};  
static_assert(std::meta::info() == std::meta::info());  
static_assert(std::meta::info() != ^^S);
```

Meta Library

<meta> provides lots of consteval-only functions for reflections.

```
namespace std::meta {  
    consteval auto identifier_of(info r) -> string_view;  
    consteval auto display_string_of(info r) -> string_view;  
    consteval auto source_location_of(info r) -> source_location;  
  
    consteval auto type_of(info r) -> info;  
    consteval auto parent_of(info r) -> info;  
    consteval auto object_of(info r) -> info;  
    consteval auto constant_of(info r) -> info;  
    consteval auto template_of(info r) -> info;  
    consteval auto template_arguments_of(info r) -> vector<info>;  
  
    // ...  
}
```

Meta Library

Example: Print all members of given class.

```
using namespace std::meta;

constexpr auto ctx = access_context::current();
template for (constexpr auto x : define_static_array(members_of(^^S, ctx))) {
    if constexpr (has_identifier(x)) { // must use constexpr-if here
        constexpr auto identifier = identifier_of(x);
        constexpr auto type_name = display_string_of(type_of(x));
        std::cout << identifier << ": " << type_name << std::endl;
    } else {
        std::cout << display_string_of(x) << std::endl;
    }
}
```

When using `if`, `identifier_of(x)` will raise an error if `x` has no identifier.

Meta Library

Example: Print all members of given class.

```
struct S {  
    int a;  
    const int b;  
    S *c;  
    S() noexcept;  
private:  
    int d;  
};
```

```
a: int  
b: const int  
c: S*  
S::S()  
constexpr S::S(const S&)  
S& S::operator=(const S&)  
constexpr S::S(S&&)  
S& S::operator=(S&&)  
constexpr S::~~S()
```

How to get private members?

Use `std::meta::access_context::unchecked()`.

define_aggregate

define_aggregate takes the reflection of an **incomplete** class/struct/union type and a range of reflections of **data member descriptions**, to **complete the definition** of given type.

define_aggregate **must** be called in **constexpr block**.

data_member_spec takes an **reflection of type** and **options**, including member name, bit width, alignments and no_unique_address flag.

```
template <class T> struct S;
constexpr {
    define_aggregate(^S<int>, {
        data_member_spec(^int,
            {.name="a", .alignment=64}),
        data_member_spec(^int, {.name="b"}),
    });
}
```

```
template <class T> struct S;
template <> struct S<int> {
    alignas(64) int a;
    int b;
};
```

define_aggregate

Example: A Simple Tuple

```
template <class ...Ts> struct Tuple {
    struct Impl;
    constexpr { define_aggregate(^^Impl, {data_member_spec(^^Ts)...}); }
    Impl impl;
    Tuple(Ts &&...args): impl{std::forward<Ts>(args)...} {}
};

constexpr auto get_nth_field(info r, std::size_t n) {
    return nonstatic_data_members_of(r, access_context::current())[n];
}

template <std::size_t I, typename ...Ts> auto &get(Tuple<Ts...> &t) {
    return t.impl.[:get_nth_field(^^decltype(t.impl), I):];
}
```

Reflection with Types and Templates

Reflection with cv-Qualifiers and References

```
static_assert(^int == ^int);  
static_assert(^int != ^const int);  
static_assert(^int != ^int &);
```

Reflection with Types and Templates

Reflection with cv-Qualifiers and References

```
static_assert(^int == ^int);  
static_assert(^int != ^const int);  
static_assert(^int != ^int &);
```

Reflection of Alias

```
using Alias = int;  
static_assert(^int != ^Alias);  
static_assert(^int == dealias(^Alias));  
static_assert(^Alias != dealias(^Alias));  
  
static_assert(^std::remove_cv_t<const int> != ^int);  
static_assert(dealias(^std::remove_cv_t<const int>) == ^int);
```

Reflection with Types and Templates

Filling Template Arguments

substitute returns a reflection for **template** with the given arguments. Substituting a **concept** will get a reflection of bool constant.

```
constexpr auto r = substitute(^std::vector, std::vector{^int});  
using T = [:r:]; // T is std::vector<int>
```

Template is only substituted, not instantiated.

```
template <typename T> struct S { typename T::X x; };  
constexpr auto r = substitute(^S, std::vector{^int}); // Okay.  
typename[:r:] si; // Error: T::X is invalid for T = int.
```

You can use **can_substitute** to check whether the substitution is valid. When it returns false, the substitution is **ill-formed**.

Reflection with Types and Templates

Extract Values from Reflection

extract<T> returns a value that has type T. Similar to `[ : : ]`, but its operand **does not need to be a constexpr**.

```
extract<int>(reflect_constant(42));           // 42
extract<bool>(substitute(^std::is_const_v, {^const int})); // true
```

```
struct S { int x; };
int S::* mptr = extract<int S::*>(^S::x);
```

Reflection with Types and Templates

Example: Implement `make_integer_sequence` with reflection.

```
template <typename T>
constexpr std::meta::info make_integer_sequence_impl(T N) {
    std::vector args{^T};
    for (T k = 0; k < N; ++k) {
        args.push_back(std::meta::reflect_constant(k));
    }
    return substitute(^std::integer_sequence, args);
}

template <typename T, T N>
using make_integer_sequence = [:make_integer_sequence_impl<T>(N):];

// std::integer_sequence<int, 0, 1, 2, 3>
make_integer_sequence<int, 4>{};
```

Compile-Time Strings

Before C++ 20, we cannot take a string literal as a non-type template argument directly:

```
template <const char *Str> struct S {};  
using T = S<"abc">;          // Error: string literals can never be used  
  
static constexpr char s[] = "abc";  
using U = S<s>;               // Ok
```

Compile-Time Strings

Before C++ 20, we cannot take an string literal as a non-type template argument directly:

```
template <const char *Str> struct S {};  
using T = S<"abc">;          // Error: string literals can never be used
```

```
static constexpr char s[] = "abc";  
using U = S<s>;              // Ok
```

In C++ 20, we can use **structural types** as non-type template parameters.

```
template <size_t N> struct static_string {  
    char data[N];  
    constexpr static_string(const char (&str)[N])  
        { std::copy_n(str, N, data); }  
};  
template <static_string Str> struct S {};  
using T = S<"abc">;
```

Compile-Time Strings

C++ 26 introduced `std::define_static_string` to define a compile-time string.

```
template <const char *Str> struct S {};  
using T = S<std::define_static_string("test")>;
```

`std::define_static_array` and `std::define_static_object` is also introduced to create compile time objects.

```
// A fairly awkward implementation for define_static_string.  
constexpr auto define_static_string(string_view str) -> char const* {  
    return define_static_array(views::concat(str, views::single('\0'))).data();  
}
```

Compile-Time Strings

Example: Convert snake_case to PascalCase in compile time.

```
constexpr char toupper(char c) {  
    return (c >= 'a' && c <= 'z') ? (c - 'a' + 'A') : c;  
}  
  
constexpr auto snake_to_pascal_case(std::string_view identifier) {  
    return define_static_string(identifier |  
        views::split("_"sv) |  
        views::filter([](auto &&token) { return token.size() > 0; }) |  
        views::transform([](auto &&token) {  
            return views::concat(views::single(toupper(token[0])),  
                token | views::drop(1));  
        }) |  
        views::join);  
}
```

Annotations

An annotation is a **compile-time value** associated to a entity.

```
struct Foo {};
```

```
struct C {  
    [[=1]] int a;  
    [[=Foo(), =42]] int b;  
};
```

Meta library provides `annotations_of`, `annotations_of_with_type` and `is_annotation` for queries and checks.

```
constexpr auto r = ^^C::a;  
constexpr auto anno = std::meta::annotations_of(r)[0];  
static_assert(std::meta::type_of(anno) == ^^int);  
static_assert(std::meta::extract<int>(anno) == 1);  
static_assert(std::meta::is_annotation(anno));
```

Annotations

Difference to **Attributes**: Attributes are designed with open grammar, and can be **ignored by implementation**.

```
[[nodiscard, gnu::always_inline]]  
[[deprecated("don't use me")]]    // Might be ignored when reflecting  
[[=Annotation()]]  
void f();
```

Difference to **Decorators**: Decorators extends the behavior of functions and classes. Annotations are just compile-times values and **cannot change the code**.

```
def my_decorator(f):  
    def wrapper(*args, **kwargs):  
        result = f(*args, **kwargs)  
        return result  
    return wrapper  
  
@my_decorator  
def greet(name):  
    print(f"Hello, {name}!")
```

Outline

Introduction	1	Limited Code Generation	28
Language and Library Features	3	Compile-Time Variables	30
Expansion Statement (Template-For)	4	Applications	31
“Cat-Ears” Operator (^)	6	Parsing Command Lines	32
Splicers ([: :])	10	Serialization and Deserialization	33
std::meta::info	12	MVVM in C++	34
Meta Library	13		
define_aggregate	15		
Reflection with Types and Templates . .	17		
Compile-Time Strings	21		
Annotations	24		
Compile-Time Programming	26		
constexpr and constexpr	27		

constexpr and consteval

constexpr specifier is to declare **functions** that always produce a **compile-time expression**.

constexpr is used to declare a compile-time variable or a function that **can be evaluated in compile-time**.

```
constexpr int foo(int x) {  
    return 2 * x;  
}  
int x = foo(10);  
int y = foo(x);
```

```
constexpr int foo(int x) {  
    return 2 * x;  
}  
int x = foo(10);  
int y = foo(x);    // Error!
```

Variables in consteval functions **may not** be constexpr, though its return value is constexpr.

```
constexpr int foo(int x) {  
    // constexpr int v = x; // Error!  
    return 2 * x;  
}
```

```
template <int I, int J> struct S {};  
constexpr int x = foo(10);  
using T = S<x, foo(20)>;
```

Limited Code Generation

With `define_aggregate` and annotations, we can create a wrapper to generate codes.

```
template <std::meta::info PropertyMeta>
class PropertyProxy {
    using T = typename [:std::meta::type_of(PropertyMeta):];
    using Object = typename [:std::meta::parent_of(PropertyMeta):];
    Object &object;
public:
    PropertyProxy(Object &object): object(object) {}
    T &get() const noexcept { return object->[:PropertyMeta:]; }
    T &set(const T &value) { return object->[:PropertyMeta:] = value; }
};

struct S { int x; } obj;
PropertyProxy<^^S::x> x_proxy(obj);
```

Limited Code Generation

Unlike decorators, we cannot directly modify the code itself. A function or a class wrapper is always needed.

```
[ [=OnEvent("EventName")] ]  
void event_handler();
```

```
constexpr { /* Process annotations... */ }
```

Currently we can only define a **struct** with **public data members**. `define_class` with more features might be introduced in the future.

Compile-Time Variables

Currently, we can use variables in `constexpr` environment. But we cannot declare a `constexpr`-only variable for compile time.

The following example shows a compile-time counter.

```
template <int N> struct Helper;
struct Counter {
    static constexpr int latest() {
        int k = 0;
        while (is_complete_type(substitute(Helper, {reflect_constant(k)}))) ++k;
        return k;
    }
    static constexpr void increment() {
        define_aggregate(substitute(Helper, {reflect_constant(latest())}), {});
    }
};
```

Compile-Time Variables

Currently, we can use variables in `constexpr` environment. But we cannot declare a `constexpr`-only variable for compile time.

The following example shows a compile-time counter.

```
constexpr int x = Counter::latest(); // x initialized to 0.
```

```
constexpr { Counter::increment(); }
```

```
constexpr int y = Counter::latest(); // y initialized to 1.
```

```
constexpr { Counter::increment(); }
```

```
constexpr int z = Counter::latest(); // z initialized to 2.
```

```
static_assert(x == 0);
```

```
static_assert(y == 1);
```

```
static_assert(z == 2);
```

Outline

Introduction	1	Limited Code Generation	28
Language and Library Features	3	Compile-Time Variables	30
Expansion Statement (Template-For)	4	Applications	31
“Cat-Ears” Operator (^)	6	Parsing Command Lines	32
Splicers ([: :])	10	Serialization and Deserialization	33
std::meta::info	12	MVVM in C++	34
Meta Library	13		
define_aggregate	15		
Reflection with Types and Templates . .	17		
Compile-Time Strings	21		
Annotations	24		
Compile-Time Programming	26		
constexpr and constexpr	27		

Parsing Command Lines

```
template <typename Options>
Options parse_options(const std::string &args) {
    Options options;
    constexpr auto ctx = access_context::current();
    template for (constexpr auto member :
        define_static_array(nonstatic_data_members_of(^Options, ctx))) {
        // do something ...
    }
    return options;
}

struct CommandOptions {
    std::string file_name = "input.txt"; // Option "--file_name <string>"
    [[=verify::Max(10), =verify::Min(0)]] // Annotations for verification.
    int count = 1;                       // Option "--count <int>"
};
```

Serialization and Deserialization

An example behaves like Rust's `serde` library.

```
struct [[=serde::derive]] Person {  
    [[=serde::rename("first name")]] std::string first;  
    [[=serde::rename("last name")]] std::string last;  
};  
  
// {"first name":"Peter","last name":"Dimov"}  
std::cout << boost::json::value_from(Person{.first="Peter", .last="Dimov"});
```

MVVM in C++

```
class ViewModel {  
    [[=ObservableProperty()]] int property = -1;  
  
public:  
    [[=RelayCommand()]] void TestCommand() { std::println("Test"); }  
  
    bool CanExecuteAnotherCommand() { return property == 42; }  
  
    [[=mvvm::RelayCommand(^CanExecuteAnotherCommand)]]  
    void AnotherCommand() { std::println("Another"); }  
  
    void OnPropertyChanged(int value) {}  
};  
  
ObservableObject<ViewModel> vm; vm->property = 42; vm->AnotherCommand();
```

Outline

Introduction	1	Limited Code Generation	28
Language and Library Features	3	Compile-Time Variables	30
Expansion Statement (Template-For)	4	Applications	31
“Cat-Ears” Operator (^)	6	Parsing Command Lines	32
Splicers ([: :])	10	Serialization and Deserialization	33
std::meta::info	12	MVVM in C++	34
Meta Library	13		
define_aggregate	15		
Reflection with Types and Templates . .	17		
Compile-Time Strings	21		
Annotations	24		
Compile-Time Programming	26		
constexpr and constexpr	27		

Modern C++ Topics: Static Reflection

Thanks for Your Listening!

References:

- [P1036R5] Expansion Statements
- [P2996R13] Reflection for C++26
- [P3289R1] `constexpr` blocks
- [P3491R3] `define_static_{string,object,array}`
- [P3394R4] Annotations for Reflection